

Experiment No.: 2

Aim

Create a Blockchain using Python

Lab Objectives

- To understand the basic concept of Blockchain.
- To implement a simple public/private Blockchain structure using Python.
- To implement Proof-of-Work mining.
- To implement transactions in a Blockchain.
- To validate the integrity of the Blockchain using cryptographic hashing.
- To expose Blockchain functionality through a Flask Web API.

Problem Statement

Create a basic Blockchain using Python and Flask. The Blockchain should support:

1. Creation of a Genesis Block.
2. Creation of transactions.
3. Mining of blocks using Proof-of-Work.
4. Mining only when transactions are available.
5. Removal of transactions from the pending transaction list before mining.
6. Proof-of-Work with a cryptographic puzzle requiring **000 leading zeros** in the hash.
7. Displaying the complete Blockchain.
8. Validating the Blockchain.
9. Providing Flask API endpoints for interacting with the Blockchain.

Theory

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains information such as transactions, timestamp, proof, and the hash of the previous block.

Each block is connected to the previous block using its cryptographic hash. This makes it difficult to modify previously stored data without affecting the entire chain.

Main Concepts

1. Block Creation

A block contains:

- Index
- Timestamp
- Transactions
- Proof
- Previous block hash

The first block in the Blockchain is called the **Genesis Block**.

2. Transactions

Transactions represent data that needs to be stored in the Blockchain.

Transactions are first stored in a pending transaction list. When a miner performs mining, these transactions are added to a new block.

3. Proof-of-Work

Proof-of-Work is a computational puzzle used for mining.

In this experiment, the miner searches for a number called a **nonce/proof** such that the SHA-256 hash satisfies:

000xx

That means the hash must contain **three leading zeros**.

4. Hashing

SHA-256 is used to generate the cryptographic hash of a block.

A small change in the block data produces a completely different hash.

5. Blockchain Validation

The Blockchain is considered valid only when:

- Every block correctly references the previous block.
- The Proof-of-Work condition is satisfied.
- The chain structure has not been tampered with.

6. Flask Web API

Flask provides API endpoints for interacting with the Blockchain.

The following endpoints are implemented:

Endpoint	Method	Purpose
/mine_block	GET	Mine a new block
/add_transaction	POST	Add a new transaction
/get_chain	GET	Display the complete Blockchain
/is_valid	GET	Check Blockchain validity

Tools and Libraries Used

- Python
- Google Colab
- Flask
- Postman
- datetime
- hashlib
- json
- threading
- requests

Complete Program

```
# =====  
  
# EXPERIMENT NO. 2  
  
# CREATE A BLOCKCHAIN USING PYTHON  
  
# Google Colab Ready  
  
# =====
```

```
# Install Flask if required
```

```
!pip -q install flask
```

```
# =====
```

```
# IMPORT REQUIRED LIBRARIES
```

```
# =====
```

```
import datetime
```

```
import hashlib
```

```
import json
```

```
import threading
```

```
import time
```

```
from flask import Flask, jsonify, request
```

```
# =====
```

```
# BLOCKCHAIN CLASS
```

```
# =====
```

```
class Blockchain:
```

```
    def __init__(self):
```

```
        # Stores all blocks
```

```
        self.chain = []
```

```
        # Stores pending transactions
```

```
        self.transactions = []
```

```
        # Create Genesis Block
```

```

self.create_block(

    proof=1,

    previous_hash='0',

    transactions=[]

)

# =====

# CREATE BLOCK

# =====

def create_block(self, proof, previous_hash, transactions):

    block = {

        'index': len(self.chain) + 1,

        'timestamp': str(datetime.datetime.now()),

        'transactions': transactions,

        'proof': proof,

        'previous_hash': previous_hash

    }

    self.chain.append(block)

    return block

# =====

# GET PREVIOUS BLOCK

# =====

```

```

def get_previous_block(self):

    return self.chain[-1]

# =====

# CREATE TRANSACTIONS

# =====

def create_transactions(self, sender, receiver, amount):

    transaction = {

        'sender': sender,

        'receiver': receiver,

        'amount': amount

    }

    self.transactions.append(transaction)

    # Return index of the block that will contain

    # this transaction

    return len(self.chain) + 1

# =====

# PROOF OF WORK

# =====

def proof_of_work(self, previous_proof):

    new_proof = 1

    while True:

```

```

# Cryptographic puzzle

hash_operation = hashlib.sha256(

    str(new_proof ** 2 - previous_proof ** 2).encode()

).hexdigest()

# Required condition:

# Hash must contain 000 leading zeros

if hash_operation.startswith('000'):

    return new_proof

    new_proof += 1


# =====

# HASH A BLOCK

# =====

def hash(self, block):

    encoded_block = json.dumps(

        block,

        sort_keys=True

    ).encode()

    return hashlib.sha256(encoded_block).hexdigest()


# =====

# MINE BLOCK

# =====

```

```

def mine_block(self):

    # -----

    # IMPORTANT:

    # Mine ONLY if transaction list is not empty

    # -----

    if len(self.transactions) == 0:

        return None

    # Get previous block

    previous_block = self.get_previous_block()

    # Get previous proof

    previous_proof = previous_block['proof']

    # Find Proof-of-Work solution

    proof = self.proof_of_work(previous_proof)

    # Get hash of previous block

    previous_hash = self.hash(previous_block)

    # -----

    # Remove/copy transactions before mining

    # -----

    transactions_to_mine = self.transactions.copy()

    # Clear pending transaction list

    self.transactions.clear()

```

```
# -----
```

```
# Create new block
```

```
# -----
```

```
block = self.create_block(  
    proof=proof,  
    previous_hash=previous_hash,  
    transactions=transactions_to_mine  
)  
  
return block
```

```
# =====
```

```
# VALIDATE BLOCKCHAIN
```

```
# =====
```

```
def is_chain_valid(self, chain):  
  
    if len(chain) == 0:  
        return False  
  
    previous_block = chain[0]  
  
    block_index = 1  
  
    while block_index < len(chain):  
        block = chain[block_index]  
  
# -----
```

```

# Check previous hash

# -----

if block['previous_hash'] != self.hash(previous_block):

    return False

# -----

# Validate Proof-of-Work

# -----

previous_proof = previous_block['proof']

proof = block['proof']

hash_operation = hashlib.sha256(

    str(proof ** 2 - previous_proof ** 2).encode()

).hexdigest()

# Hash must have 000 leading zeros

if not hash_operation.startswith('000'):

    return False

previous_block = block

block_index += 1

return True

# =====

# CREATE BLOCKCHAIN OBJECT

```

```

# =====

blockchain = Blockchain()

# =====

# FLASK APPLICATION

# =====

app = Flask(__name__)

# =====

# ADD TRANSACTION API

# =====

@app.route('/add_transaction', methods=['POST'])

def add_transaction():

    data = request.get_json()

    required_fields = [

        'sender',

        'receiver',

        'amount'

    ]

    # Check required fields

    for field in required_fields:

        if field not in data:

```

```
    return jsonify({  
        'message': f'Missing field: {field}'  
    }), 400
```

```
# Create transaction
```

```
block_index = blockchain.create_transactions(  
    sender=data['sender'],  
    receiver=data['receiver'],  
    amount=data['amount']  
)
```

```
response = {  
    'message': 'Transaction added successfully',  
    'transaction': {  
        'sender': data['sender'],  
        'receiver': data['receiver'],  
        'amount': data['amount']  
    },  
    'block_index': block_index  
}
```

```
return jsonify(response), 201
```

```
# =====
```

```

# MINE BLOCK API

# =====

@app.route('/mine_block', methods=['GET'])

def mine_block():

    # -----

    # Mine only when transactions are available

    # -----

    if len(blockchain.transactions) == 0:

        return jsonify({

            'message': 'No transactions available. Add a transaction before mining.'

        }), 400

    # Mine the block

    block = blockchain.mine_block()

    response = {

        'message': 'Congratulations, you just mined a block!',

        'index': block['index'],

        'timestamp': block['timestamp'],

        'transactions': block['transactions'],

        'proof': block['proof'],

        'previous_hash': block['previous_hash']

    }

```

```
return jsonify(response), 200
```

```
# =====
```

```
# GET COMPLETE BLOCKCHAIN API
```

```
# =====
```

```
@app.route('/get_chain', methods=['GET'])
```

```
def get_chain():
```

```
    response = {
```

```
        'chain': blockchain.chain,
```

```
        'length': len(blockchain.chain),
```

```
        'pending_transactions': blockchain.transactions
```

```
    }
```

```
    return jsonify(response), 200
```

```
# =====
```

```
# VALIDATE BLOCKCHAIN API
```

```
# =====
```

```
@app.route('/is_valid', methods=['GET'])
```

```
def is_valid():
```

```
    valid = blockchain.is_chain_valid(
```

```
        blockchain.chain
```

```
    )
```

```
if valid:
```

```
    response = {
```

```
        'message': 'All good. The Blockchain is valid.'
```

```
    }
```

```
else:
```

```
    response = {
```

```
        'message': 'Houston, we have a problem. The Blockchain is not valid.'
```

```
    }
```

```
return jsonify(response), 200
```

```
# =====
```

```
# RUN FLASK SERVER IN GOOGLE COLAB
```

```
# =====
```

```
def run_server():
```

```
    app.run(
```

```
        host='0.0.0.0',
```

```
        port=7000,
```

```
        debug=False,
```

```
        use_reloader=False
```

```
    )
```

```
# Start Flask in a background thread
```

```
server_thread = threading.Thread(  
    target=run_server,  
    daemon=True  
)  
  
server_thread.start()  
  
# Give the server some time to start  
  
time.sleep(3)  
  
print("=====  
print("BLOCKCHAIN SERVER STARTED")  
print("=====  
print("\nAvailable API Endpoints:")  
print("\n1. Add Transaction")  
print("POST http://127.0.0.1:7000/add_transaction")  
print("\n2. Mine Block")  
print("GET http://127.0.0.1:7000/mine_block")  
print("\n3. Get Blockchain")  
print("GET http://127.0.0.1:7000/get_chain")  
print("\n4. Check Blockchain Validity")  
print("GET http://127.0.0.1:7000/is_valid")  
print("\n=====  
print("Server is running...")
```

```
print("=====")
```

Testing the Program

Step 1: Start the Program

Run the complete Python code in Google Colab.

The Flask server will start on:

<http://127.0.0.1:7000>

Step 2: Add a Transaction

Use Postman:

POST /add_transaction

Request Body:

```
{  
    "sender": "Rutuja",  
    "receiver": "Isha",  
    "amount": 700  
}
```

Expected response:

```
{  
    "block_index": 2,  
    "message": "Transaction added successfully",  
    "transaction": {  
        "amount": 700,
```

```
    "receiver": "Isha",  
    "sender": "Rutuja"  
  }  
}
```

Step 3: Mine the Block

Send:

GET /mine_block

The system performs Proof-of-Work and searches for a proof whose hash starts with:

000

Expected response structure:

```
{  
  "message": "Congratulations, you just mined a block!",  
  "index": 2,  
  "timestamp": "2026-08-27 13:00:00",  
  "transactions": [  
    {  
      "amount": 700,  
      "receiver": "Isha",  
      "sender": "Rutuja"  
    }  
  ],  
  "proof": 533,
```

```
"previous_hash": "..."  
}
```

The actual proof value and hash will be different because they are calculated during execution.

Step 4: Get the Complete Blockchain

Send:

GET /get_chain

The response displays:

- Complete Blockchain
- Number of blocks
- Pending transactions

Example:

```
{  
  "chain": [  
    {  
      "index": 1,  
      "timestamp": "...",  
      "transactions": [],  
      "proof": 1,  
      "previous_hash": "0"  
    },  
    {  
      "index": 2,  
      "timestamp": "...",
```

```
"transactions": [  
  {  
    "sender": "Rutuja",  
    "receiver": "Isha",  
    "amount": 700  
  }  
],  
"proof": 533,  
"previous_hash": "..."  
}  
],  
"length": 2,  
"pending_transactions": []  
}
```

Step 5: Validate the Blockchain

Send:

GET /is_valid

Expected response:

```
{  
  "message": "All good. The Blockchain is valid."  
}
```

Important Observation

If `/mine_block` is called without adding a transaction first, the Blockchain will **not mine a new block**.

Expected response:

```
{  
  
  "message": "No transactions available. Add a transaction before mining."  
  
}
```

This satisfies the requirement that the block should be mined **only when the transaction list is not empty**.

Algorithm

Blockchain Creation

1. Initialize an empty Blockchain.
2. Create the Genesis Block.
3. Maintain a list of pending transactions.
4. Add transactions using `create_transactions()`.
5. Check whether transactions are available.
6. Get the proof of the previous block.
7. Execute Proof-of-Work.
8. Search for a proof whose SHA-256 hash starts with `000`.
9. Calculate the hash of the previous block.
10. Copy the pending transactions.
11. Clear the pending transaction list.
12. Create and add the new block.
13. Validate the Blockchain by checking previous hashes and Proof-of-Work.

Output

The program successfully:

- Created a Genesis Block.
- Added transactions.
- Prevented mining when no transactions were available.
- Performed Proof-of-Work mining.
- Searched for a hash with **000 leading zeros**.
- Added transactions to a mined block.
- Cleared pending transactions after mining.
- Displayed the complete Blockchain.
- Verified Blockchain validity using Flask APIs.

Conclusion

We successfully created a basic Blockchain using Python and Flask. The implementation demonstrates important Blockchain concepts such as block creation, transaction management, SHA-256 hashing, Proof-of-Work mining, and Blockchain validation. The `create_transactions()` method allows transactions to be stored before mining, while the mining process operates only when pending transactions are available. The Proof-of-Work algorithm searches for a valid proof whose hash contains **000 leading zeros**, demonstrating the computational nature of Blockchain mining. Flask API endpoints were also implemented to add transactions, mine blocks, retrieve the Blockchain, and verify its validity. Overall, the experiment provides a practical understanding of the fundamental concepts and working of Blockchain technology.